

# Week 10 – Telemetry, Responsive Dashboard & HTTPS Profiling Report

**Project:** Phase-2 Backend / Data Analysis Project

**Student:** Akshay

**Environment:** Windows, Node.js v24.11.1

## 1. Weekly Objective

Build a responsive dashboard interface that consumes telemetry data streams dynamically, processes telemetry packets through a worker/consumer pipeline, exposes custom Prometheus metrics, and verifies HTTPS, rate limiting, browser cache policy, and stress-test behavior.

## 2. Implemented Components

- Telemetry consumer polling the application log for newly appended telemetry packets.
- Real-time speed index calculation using  $1000 / \text{latency}(\text{ms})$ .
- Prometheus `telemetry_speed_index` gauge with `event_type` labels.
- Worker event, queue, processing-duration, stress-latency, and peak-latency metrics.
- Responsive telemetry dashboard with metric cards and time-series charts.
- API rate limiter: 100 requests per 60 seconds under `/api`.
- No-cache/no-store browser policy for `telemetry.html`.
- Local HTTPS server using a self-signed localhost certificate.

## 3. HTTPS / SSL Verification

A localhost RSA-2048 self-signed certificate and private key were generated successfully. The Node.js server was changed from HTTP to an HTTPS server using the certificate files. Browser access to `https://localhost:3000` succeeded after accepting the expected self-signed certificate warning.

Verified server output included: **Server running on https://localhost:3000, Connected to SQLite database., and Redis connection ready.**

## 4. Telemetry Stress Verification

| Packet | Latency (ms) | Speed Index |
|--------|--------------|-------------|
| 1      | 373          | 2.68        |
| 2      | 50           | 20.00       |
| 3      | 959          | 1.04        |
| 4      | 200          | 5.00        |
| 5      | 335          | 2.99        |

Five `stress_burst` telemetry packets were successfully processed. The latest Prometheus speed-index value was 2.99, matching packet 5. The recorded peak latency was 959 ms, and `worker_events_processed_total` for `stress_burst` reached 5.

## 5. Metrics Verification

| Metric | Verified result |
|--------|-----------------|
|--------|-----------------|

|                                                                  |                    |
|------------------------------------------------------------------|--------------------|
| telemetry_speed_index{event_type="stress_burst"}                 | 2.99               |
| worker_events_processed_total{event_type="stress_burst"}         | 5                  |
| stress_test_peak_latency_milliseconds{event_type="stress_burst"} | 959                |
| worker_queue_size                                                | 0 after processing |

## 6. Rate Limiting and Cache Policy

The API limiter was configured for 100 requests per 60-second window. Previous verification showed requests to /api/products reaching HTTP 429 after the limit was exceeded. The telemetry page was configured with Cache-Control: no-cache, no-store, must-revalidate, plus Pragma: no-cache and Expires: 0 headers.

## 7. Performance / Resource Observations

| Measurement                   | Observed result                 |
|-------------------------------|---------------------------------|
| Baseline working set          | ≈65.01 MB                       |
| After 1000-event stress       | ≈66.27 MB                       |
| Working-set increase          | ≈1.26 MB                        |
| 50 /metrics requests          | 1.319 s total; ≈26.4 ms/request |
| 1000-event stress API request | ≈113.75 ms                      |
| Worker processing average     | ≈0.95 ms/event                  |
| Peak simulated stress latency | 1000 ms in prior stress audit   |

## 8. Final Verification Summary

The Week 10 implementation successfully demonstrates telemetry ingestion, real-time speed-index calculation, Prometheus metric exposure, responsive dashboard support, API rate limiting, browser cache controls, and local HTTPS. SQLite and Redis remained connected during verification. The stress-test pipeline processed five telemetry packets and produced matching metric values.

**Conclusion:** Week 10 verification requirements were substantially completed and the measured results provide a reproducible basis for the profiling and verified-check summary.